

SILVERA Avi
PUCHOIS Ambrosia
RADJABALY Ilyan
REMILA Lydia

Note technique de la démarche de développeur

A l'intention de l'ami.e informaticien.ne

TABLE DES MATIÈRES

<u>I. Code VBA</u> - Préparation d'un fichier propre contenant obligations et actions.....	2
<u>II. Code SQL</u> - Création d'une base de données à partir du fichier précédent.....	6
<u>III. Code VBA</u> - Création de l'outil de gestion de l'investissement.....	8

Ce projet est structuré en trois parties. Premièrement, nous avons créé un fichier “propre” contenant les données constituant l’univers d’investissement à l’aide d’un code VBA qui regroupe toutes les actions et indices disponibles (et qui proviennent de l’archive de données fournie avec le projet) ainsi que les 10 obligations (dont la moitié avec des coupons semi-annuels), que nous avons générées aléatoirement. Dans un deuxième temps, nous avons créé une base de données SQL qui reprend le fichier de données “nettoyées” créé à l’étape précédente. Enfin, nous avons créé un nouveau code VBA afin de générer l’outil de gestion d’investissement que notre cliente va pouvoir utiliser. Ce dernier est accompagné d’une notice d’utilisation.

I. Code VBA - Préparation d’un fichier propre contenant obligations et actions

Notre cliente dispose de deux poches d’investissements: elle peut investir dans des actions ou dans des obligations. L’ensemble de ces produits constitue l’univers d’investissement.

Dans cette première partie, nous allons faire l’acquisition des différentes données concernant les actions qui nous ont été fournies, et nous allons générer 10 obligations, puis les regrouper dans un seul et même classeur Excel, sur deux feuilles différentes. Nous exporterons ensuite ces deux feuilles Excel sous le format CSV, afin de pouvoir les utiliser pour créer notre base de données SQL. Nous disposons pour ce faire de 4 classeurs Excel “FrenchStocks.xlsx”, “Indexes.xlsx”, “Rates.xlsx”, et “TechStocks.xlsx”. Nous considérons ici que notre riche cliente a le choix d’investir directement dans les actions FrenchStocks et TechStocks, ainsi que dans 10 obligations que nous allons créer. Elle peut également investir dans un indice, mais attention, chaque indice compte pour un seul produit. Nous verrons par la suite au moment de la création du portefeuille, qu’elle doit au moins investir la moitié de sa richesse en actions. L’ensemble de ces données constituant l’univers d’investissement sont agrégées dans un unique classeur Excel nommé “UniversInvestissement.xlsm”.

Nous avons fait le choix de coder la base de données comme nous l’avons fait en ayant à l’esprit l’idée de pouvoir avoir une base qui soit facilement exportable dans SQL et facilement exploitable pour la dernière partie du projet.

Notre code principal est situé dans le module “BaseDonnees”.

Nous avons commencé par la déclaration de toutes les variables nécessaires pour manipuler les feuilles de calcul, les classeurs, les chemins d’accès, les compteurs et les paramètres des obligations. Nous avons au préalable créé un module de classe “ObligationClasse” afin de pouvoir manipuler les objets obligations dans ce code.

Module de classe ObligationClasse

Ce module de classe définit une nouvelle classe objet VBA pour représenter une obligation. Nous commençons par lui définir des attributs: sa maturité, sa valeur nominale, son taux de coupon, la fréquence de paiement des coupons et le taux sans risque. Le poids de l'obligation dans le portefeuille sera déterminé dans la troisième partie de ce projet. Nous utilisons un constructeur Initiation_Obligation qui sert à initialiser les caractéristiques de l'obligation au moment de la création de l'objet. Il permet d'attribuer automatiquement à l'obligation ses paramètres clés : la maturité, le nominal, le taux de coupon, la périodicité et le taux sans risque. Ainsi il nous permet de préparer l'objet à être utilisé pour effectuer des calculs (valeur, duration,...) en lui donnant toutes les informations nécessaires dès le départ.

- Nous calculons par la suite la valeur des coupons versés à l'aide de Property Get... qui permet d'accéder à une valeur calculée comme si c'était un attribut de l'objet, mais sans avoir besoin d'appeler une fonction explicite. Nous avons calculé le coupon selon la formule suivante:

$$Coupon = Nominal * ((1 + TauxCoupon_{annuel})^{\frac{1}{periodicite}} - 1)$$

Cette formule permet de convertir un taux annuel en un taux périodique équivalent avec capitalisation composée, puis à calculer le montant du coupon pour une période.

- Toujours à l'aide de Property Get ... nous calculons la valeur de l'obligation, et pour ce faire, il faut sommer les flux de trésorerie actualisés de l'obligation, c'est-à-dire les coupons versés et le remboursement final à la date de l'achat. Nous utilisons la formule suivante:

$$V = \sum_{t=1}^N \frac{Coupon}{(1 + TauxSansRisqueIni)^t} + \frac{Nominal}{(1 + TauxSansRisqueIni)^N}$$

Pour implémenter cette somme en VBA, il est nécessaire de passer par une boucle. Pour ce faire, nous commençons par initialiser la valeur de l'obligation à 0, puis à chaque itération, donc à chaque paiement de coupon, nous actualisons le montant du coupon et nous l'ajoutons au total. Une fois la dernière itération atteinte, à maturité, nous ajoutons en plus la valeur nominale actualisée au total.

- La duration de Macaulay est la moyenne pondérée des échéances des flux de trésorerie d'une obligation, exprimée en années. Elle sert à mesurer le temps moyen pondéré avant de récupérer l'investissement initial. Plus la duration est élevée, plus l'obligation est sensible aux variations des taux d'intérêt.

$$D_{Macaulay} = \frac{\sum_{t=1}^{maturite} \frac{CashFlow_t}{(1 + TauxSansRisque)^t} * t}{\sum_{t=1}^{maturite} \frac{CashFlow_t}{(1 + TauxSansRisque)^t}}$$

Pour implémenter ce calcul VBA nous introduisons une variable CashFlow qui représente les flux de trésorerie générés par l'obligation (coupons + nominal à maturité) et nous initialisons la duration à 0. Le but est de faire une boucle qui parcourt chaque période de paiement des coupons (donc des semestres ou des années) et qui ajoute le montant du coupon au CashFlow à chaque itération, et ce jusqu'à maturité, où on ajoute également la valeur nominale de l'obligation. Il suffit ensuite à chaque période de diviser le CashFlow actualisé et multiplié par la période à laquelle il est généré par le CashFlow actualisé de la période. Or le dénominateur correspond déjà à la Valeur de l'obligation qu'on a déjà calculée puisque pour calculer la valeur présente d'une obligation il faut sommer les CashFlow actualisés générés par l'obligation.

- Alors que la duration de Macaulay exprime le temps moyen pondéré de récupération des flux de trésorerie, la duration modifiée ajuste cela pour mesurer directement la sensibilité du prix de l'obligation aux variations du taux d'intérêt.

$$D_{MacaulayModifié} = \frac{D_{Macaulay}}{1 + \frac{TauxSansRisque}{Periodicite}}$$

Une fois les variables ayant été déclarées et le module de classe créé, nous pouvons à présent nous intéresser au code principal. Notre but va donc être de regrouper les données sur les obligations sur une feuille, et les données sur les actions et indices sur une autre feuille, puisqu'ils présentent la même structure dans les archives que nous avons à disposition.

Nous commençons par demander à l'utilisateur d'entrer les chemins des fichiers sources (Indexes, TechStocks, FrenchStocks) à l'aide de Application.InputBox, et nous stockons ces chemins d'accès dans des variables. Nous avons choisi de procéder ainsi car c'était le plus simple étant donné que chaque utilisateur est libre de stocker ses fichiers n'importe où sur son ordinateur, et chacun peut facilement retrouver le chemin d'accès des fichiers, par exemple en accédant aux propriétés du fichier dans l'explorateur des fichiers. Puis, nous les ouvrons en lecture seule, afin de ne pas modifier nos fichiers source d'origine. De plus, nous référençons chacune de leur première feuille, puisque ce sont les seules contenant des données.

Puisqu'il nous a été demandé de créer des fichiers "propres", nous avons commencé par traiter les valeurs de prix manquantes dans les fichiers contenant les actions et indices. Il manquait les prix des premières dates de l'action OR.PA dans FrenchStocks, les prix des dernières dates de la FrenchStocks CA.PA et de l'indice N225, et les prix pour les dates du milieu de la période pour GLE.PA dans FrenchStocks. Nous avons choisi d'utiliser la méthode backward fill qui remplit les cases vides avec la valeur au-dessus, puis forward fill qui remplit les vides avec la valeur en-dessus pour s'assurer que toutes les cases vides aient bien été traitées. Puisque nous avons précédemment ouvert nos fichiers en mode lecture seule, ils ne seront pas modifiés.

Nous sommes ensuite passé à la préparation des feuilles "Obligations" et "ActionsIndices" qui constituent notre base de données, en commençant par les vider si elles existent déjà (dans le cas où nous faisons fonctionner plusieurs fois le programme), ou bien en les créant directement si elles n'existent pas déjà. Pour finir la préparation des feuilles, nous avons défini les en-têtes des colonnes.

Nous avons ensuite pu passer au remplissage de la feuille "Actions". Nous avons repris la structure globale des fichiers source, seulement en agrégeant TechStocks.xlsx, FrenchStocks.xlsx et Indexes.xlsx, puisque les trois fichiers contenaient les mêmes dates. Nous avons eu à reporter la première colonne contenant les dates de l'un des trois fichiers, compter le nombre de produits dans chacun des fichiers, (colonnes - 1), compter le nombre de dates (lignes - 1), puis à créer pour chaque fichier une boucle parcourant les colonnes, et une boucle à l'intérieur de cette boucle pour parcourir chaque date de chaque action/obligation et reporter les valeurs contenues dans les cellules.

Concernant le remplissage de la feuille obligations, nous avons généré 10 obligations de façon aléatoire, en ayant pour seule contrainte d'avoir la moitié des obligations avec coupon semi-annuel. Nous avons donc créé une boucle parcourant les 10 lignes censées contenir les informations sur les obligations, en n'oubliant pas au préalable d'initialiser le générateur de nombres aléatoires à l'aide de Randomize. Nous avons généré des maturités aléatoires entre 4 et 10 ans, des taux entre 1.5% et 6%, et après avoir défini les 5 premières obligations à coupon semi-annuel (périodicité = 2), nous avons choisi des coupons annuels pour les 5 autres, afin de respecter la contrainte imposée. De plus, nous avons arbitrairement choisi une valeur nominale de 1000 € pour chaque obligation et un taux sans risque fixé arbitrairement à 1%.

Enfin, nous avons pu dans les dernières colonnes ajouter les propriétés de chaque obligations, chacune ayant préalablement été définie comme un objet obligation de la classe ObligationClasse à l'aide de Set Oblig = New ObligationClasse, et chacun de ces objets ayant reçu ses attributs grâce à Call Oblig.Initiation_Obligation.

Ceci étant fait, nous avons ainsi pu récupérer les propriétés définies dans le module de classe, et nous avons pu les calculer pour chaque objet et les faire apparaître dans les quatre dernières colonnes Coupon, Valeur, Duration Macaulay, et Duration Macaulay Modifiée.

Nom	Nominal	Maturité	Taux coupon	Periodicité	Taux Sans Risque	Coupon	Valeur	Duration Macaulay	Duration Macaulay modifiée
Obligation 1	1000	4	2,55	2,00	0,01	884,14	7877,99	1,4700	1,4627
Obligation 2	1000	6	2,55	2,00	0,01	884,14	11215,68	1,9789	1,9691
Obligation 3	1000	4	4,36	2,00	0,01	1315,17	11250,05	1,3646	1,3578
Obligation 4	1000	5	4,43	2,00	0,01	1330,24	13896,08	1,6136	1,6056
Obligation 5	1000	9	4,31	2,00	0,01	1304,34	23316,13	2,6026	2,5897
Obligation 6	1000	10	4,15	1,00	0,01	4150,00	40211,20	5,5211	5,4664
Obligation 7	1000	10	2,53	1,00	0,01	2530,00	24867,69	5,5847	5,5294
Obligation 8	1000	10	2,16	1,00	0,01	2160,00	21363,30	5,6121	5,5565
Obligation 9	1000	7	4,04	1,00	0,01	4040,00	28114,62	4,0610	4,0208
Obligation 10	1000	8	4,82	1,00	0,01	4820,00	37804,57	4,5345	4,4896

Pour finir, nous avons “nettoyé” le fichier en centrant les cellules, en ajustant la police, en ajustant la taille des cellules (AutoFit), en mettant les dates, les prix et les taux dans des formats adéquats, en mettant en gras les en-têtes, et en supprimant toutes les feuilles autres que celles utiles. Enfin, nous avons fermé les 3 classeurs de données sans enregistrer de changements et avons mis en place une dernière instruction pour enregistrer des copies de nos feuilles “Obligations” et “ActionsIndices” au format CSV, en demandant au préalable à l'utilisateur de donner le chemin d'accès du fichier xlsx dans lequel nous travaillons, afin d'y ranger les fichiers CSV générés par ce code.

II. Code SQL - Création d'une base de données à partir du fichier précédent

Le code SQL permet de créer une base de données en SQL, de configurer des tables pour stocker les informations sur des obligations et des actions/indices, puis d'insérer des données provenant de fichiers CSV créés précédemment dans ces tables.

Création de la base de données :

La commande `CREATE DATABASE IF NOT EXISTS UniversInvestissement;` crée une nouvelle base de données appelée `UniversInvestissement`, mais uniquement si cette base de données n'existe pas déjà. Cela permet de garantir qu'une erreur ne sera pas générée si la base existe déjà.

Sélection de la base de données :

La commande `USE UniversInvestissement;` permet de spécifier que toutes les opérations suivantes (comme la création de tables et l'insertion de données) doivent être effectuées dans la base de données `UniversInvestissement` que nous venons de créer.

Suppression des tables existantes :

Les commandes `DROP TABLE IF EXISTS Obligations;` et `DROP TABLE IF EXISTS ActionsIndices;` suppriment les tables `Obligations` et `ActionsIndices` si elles existent déjà dans la base de données.

Création des tables:

La table `Obligations` est ensuite créée pour stocker des informations sur chaque obligation, comme le nom, la valeur nominale, le taux de coupon, la maturité, le taux sans risque, etc. Cette table est dotée d'une clé primaire sur le champ `Nom`, mais en réalité nous n'allons pas avoir besoin de faire appel à cette clé dans d'autres tables. Son utilité ici était plutôt de s'assurer que chaque obligation est identifiée de manière unique. La table `ActionsIndices` est conçue pour stocker les données relatives aux prix des actions et des indices financiers. Les colonnes de cette table incluent les dates et les prix des actions et indices.

Insertion des données depuis des fichiers CSV :

La commande `LOAD DATA INFILE` est utilisée pour charger les données depuis les fichiers CSV créés dans le code précédent vers les tables correspondantes dans la base de données SQL. La première commande charge les données du fichier `Obligations.csv` dans la table `Obligations`, et la deuxième commande charge les données du fichier `ActionsIndices.csv` dans la table `ActionsIndices`.

Chaque fichier CSV est séparé par des virgules (`FIELDS TERMINATED BY ','`), et chaque ligne est terminée par un retour chariot (`LINES TERMINATED BY '\r\n'`). L'option `IGNORE 1 LINES` permet de sauter la première ligne du fichier CSV (qui contient les en-têtes des colonnes).

Requêtes pour récupérer des données :

Enfin, nous réalisons deux requêtes comme demandé dans la consigne du projet. La première requête `SELECT * FROM Obligations;` récupère toutes les lignes et toutes les colonnes de la table `Obligations`, affichant ainsi toutes les informations des obligations stockées. La deuxième requête `SELECT Dates, N225, VIX FROM ActionsIndices WHERE Dates BETWEEN '2005-01-01' AND '2005-12-31';` récupère les valeurs des indices `N225` et `VIX` pour les dates situées entre le 1er janvier 2005 et le 31 décembre 2005.

III. Code VBA - Création de l'outil de gestion de l'investissement

L'outil a été pensé pour être simple d'utilisation, même pour quelqu'un qui n'a aucune base en VBA. Il est construit de manière claire : tout part d'un bouton dans Excel, qui guide l'utilisateur pas à pas depuis le choix des fichiers jusqu'à l'affichage des résultats, sans avoir besoin de toucher au code.

MODULE DE CLASSE - clsPoche : Représentation d'un compartiment du portefeuille

Le module de classe "clsPoche" représente une partie du portefeuille, soit la poche "Actions", soit la poche "Obligations". Elle stocke les données financières liées à cette poche (comme la valeur, la performance ou la volatilité) et permet d'y accéder facilement grâce à des propriétés.

Attributs privés:

- typePoche : nom de la poche ("Actions" ou "Obligations")
- montantAlloue : montant investi dans la poche.
- fichierSource : chemin du fichier utilisé pour la simulation
- dateRequete : date d'analyse
- valeur : valeur actuelle simulée
- performanceInitiale : performance depuis la mise en place
- m_performanceRecente : performance récente simulée
- volatilitéAction : volatilité estimée (pour les actions)
- m_duration : duration (pour les obligations).
- m_durationModifiée : duration modifiée, tenant compte du taux sans risque (pour les obligations)

Méthode principale:

- Init(nomPoche As String, montant As Double, pathFichier As String, d As Date)
Initialise la poche selon son type. Des valeurs simulées sont générées pour la performance et les indicateurs de risque.

Propriétés:

- Nom : retourne le nom de la poche.
- ValeurActuelle : retourne la valeur simulée.
- PerformanceTotale : performance depuis 2005 (simulée).
- PerformanceRecente : performance récente (simulée).

- Volatilité : volatilité estimée (uniquement pour les actions).
- Duration : duration (pour les obligations).
- DurationModifiée : duration ajustée aux taux d'intérêt (pour les obligations).

Rôle dans l'outil:

La classe "clsPoche" permet de modéliser séparément les différentes composantes du portefeuille, qu'il s'agisse d'actions ou d'obligations. Elle fournit les données nécessaires à l'analyse globale en simulant des caractéristiques spécifiques à chaque type de poche.

MODULE DE CLASSE - clsPortefeuille : Agrégation et synthèse du portefeuille

La classe "clsPortefeuille" gère les deux parties du portefeuille : la poche Actions et la poche Obligations. Elle s'occupe de les créer, de les initialiser avec leurs données, puis de générer un rapport résumé dans une nouvelle feuille Excel.

Attributs privés:

- Actions, obligations : objets de type clsPoche, représentant les deux poches du portefeuille.
- DateRequete : date de consultation saisie par l'utilisateur.
- ValeurInitiale : montant global investi (ici: 1.000.000 €).

Méthodes:

- Init(montant As Double, pathActions As String, pathObligations As String, dStr As String)
Initialise les deux poches :
 - 60 % pour les actions,
 - 40 % pour les obligations, en utilisant les fichiers sources fournis par l'utilisateur et la date de requête.
- GenererFeuilleSynthese()
Supprime la feuille "Rapport" existante si elle est présente, puis crée une nouvelle feuille. Elle y insère :
 - Le résumé du portefeuille (nom, valeur, performance totale et récente),
 - L'analyse du risque (volatilité des actions, duration et duration modifiée des obligations),
 - La valeur totale du portefeuille.

Rôle dans l'outil:

Cette classe coordonne l'ensemble du portefeuille en combinant les données des deux poches. Elle automatise la génération du rapport final et fournit à l'utilisateur une vue synthétique et structurée de son investissement.

MODULE 1: Création du menu utilisateur avec bouton et génération du rapport

Ce module permet de générer une feuille intitulée "Menu", ainsi qu'un bouton qui permet à l'utilisateur de facilement générer le rapport. Puis de générer le rapport en fonction de la date choisie par l'utilisatrice.

Procédures incluses :

Sub CréerMenu()

- Crée une nouvelle feuille Excel nommée "Menu".
- Vide tout son contenu éventuel.
- Ajoute un titre en haut de la feuille ("Outil de Gestion de Portefeuille").
- Crée un bouton nommé "Generer un rapport" qui déclenche la macro GenererRapport lorsque l'utilisateur clique dessus.

Sub GenererRapport()

- **Demande de la date à l'utilisateur:**
 - L'utilisateur est invité à entrer une date de consultation dans un format clair (AAAA-MM-JJ) via une boîte de saisie. Si l'utilisatrice n'entre pas de date, la macro s'arrête.
- **Récupération des fichiers:**
 - Le programme va récupérer le chemin d'accès aux fichiers ActionsIndices.csv et Obligations.csv qui doivent être rangés dans le même dossier que le classeur Excel "Outil.xlsm" actuellement utilisé, puisque la macro génère ces chemins d'accès à partir du chemin du classeur ouvert.
- **Exécution de la génération du rapport:**
 - La macro désactive la mise à jour de l'écran pour éviter le clignotement pendant l'exécution
 - Puis appelle la macro GenererRapportPortefeuille définie dans le MODULE2 avec les chemins des fichiers et la date choisie pour générer le rapport.

MODULE 2: Lancement centralisé du calcul de portefeuille

Ce module contient une unique procédure. Cette procédure permet de créer un portefeuille avec un montant initial et les données sur les actions et les obligations contenues dans nos fichiers propres, puis génère un rapport de synthèse de ce portefeuille.

Sub GenererRapportPortefeuille

- Déclaration d'une nouvelle instance d'un objet de la classe clsPortefeuille.
- Initialisation du portefeuille avec la méthode Init de la classe clsPortefeuille et les paramètres suivants:
 - Le montant total à investir (1.000.000 €)
 - Le chemin vers les fichiers Actions et Obligations
 - Une date qui peut être utilisée pour récupérer les données à une date spécifique dans nos fichiers propres
- Génération de la feuille de synthèse : La ligne Call portefeuille.GenererFeuilleSynthese appelle la méthode GenererFeuilleSynthese définie dans le module de classe clsPortefeuille. Cette méthode est responsable de la création d'une feuille de synthèse sur le portefeuille généré.

Note de fin:

Ce projet a permis de développer un outil complet pour la gestion d'un portefeuille d'investissement, structuré en trois étapes clés. Tout d'abord, un fichier "propre" a été créé en utilisant un code VBA pour regrouper les données sur les actions, indices et obligations, générant ainsi un univers d'investissement complet. Ensuite, une base de données SQL a été mise en place afin de faciliter l'exploitation de ces données, avec des tables dédiées aux obligations et aux actions/indices. Enfin, un outil de gestion d'investissement a été développé en VBA, offrant une interface utilisateur intuitive pour la consultation et l'analyse du portefeuille.

Notre cliente peut désormais utiliser cet outil, et pour faciliter son utilisation, un guide détaillé est fourni afin de l'accompagner dans chaque étape, de l'importation

des données à la génération des rapports d'analyse. Ce guide est conçu pour garantir une prise en main simple et rapide, même pour une personne n'ayant pas de connaissances préalables en VBA.